



UNIVERSITATEA „ȘTEFAN CEL MARE” DIN SUCEAVA
FACULTATEA DE INGINERIE ELECTRICĂ ȘI ȘTIINȚA
CALCULATOARELOR
CALCULATOARE DUAL

T H E I N V E N T O R Y

***Microserviciu pentru gestionarea
inventarului.***

Proiect realizat de:
Strugariu Filip-Daniel
FIESC Grupa: 3111Da An 1

Suceava 2026

1 Cuprins	
2 Descrierea aplicației	3
3 API	3
4 Tehnologii și Tool-uri utilizate.....	4
5 Structura aplicației	5
5.1 Schema bazei de date (Database schema).....	5
5.2 Structura backend-ului (Module)	6
5.3 Structura frontend-ului (Frontend app structure) ..	6
5.4 Structura repo-ului Git.....	6
6 Configurare și rulare	7
6.1 Rulare locală (Development):.....	7
6.2 Rulare producție:	8
7 Concluzii / Lecții învățate.....	8

1 Descrierea aplicației

The Inventory este o soluție digitală de tip **Full-Stack** concepută pentru optimizarea fluxurilor logistice în cadrul depozitelor de dimensiuni medii. Aplicația permite monitorizarea în timp real a stocurilor, gestionarea relațiilor cu furnizorii și automatizarea transferurilor de marfă între diferite locații fizice.

Inspirat de standardele platformei **ASSIST Learning**, proiectul pune accent pe o interfață utilizator intuitivă (**Dashboard**) și pe un backend performant capabil să gestioneze operațiuni **CRUD** complexe și validări riguroase de date.

2 API

Documentație automată: backend-ul utilizează specificația **OpenAPI (Swagger)** pentru generarea interfeței de testare.

Rute principale:

/products: Gestiunea catalogului de produse.

/warehouses: Administrarea capacităților de stocare.

/inventory: Logica de transfer și ajustare .

/suppliers: Baza de date a partenerilor comerciali.

The Inventory API 1.0.0 OAS 3.1

/openapi.json

Microservice for Inventory Management

Warehouse Management

GET	/api/warehouses	Get Warehouses	▼
POST	/api/warehouses	Create Warehouse	▼
GET	/api/warehouses/{id}	Get Warehouse	▼
PUT	/api/warehouses/{id}	Update Warehouse	▼
PATCH	/api/warehouses/{id}	Patch Warehouse	▼
DELETE	/api/warehouses/{id}	Delete Warehouse	▼

Supplier Management

GET	/api/suppliers	Get Suppliers	▼
POST	/api/suppliers	Create Supplier	▼
GET	/api/suppliers/{id}	Get Supplier	▼
PUT	/api/suppliers/{id}	Update Supplier	▼
PATCH	/api/suppliers/{id}	Patch Supplier	▼
DELETE	/api/suppliers/{id}	Delete Supplier	▼

Product Management			^
GET	/api/warehouses/{warehouseId}/products/all	Get Absolutely All Products	▼
POST	/api/warehouses/{warehouseId}/products	Create Product	▼
GET	/api/warehouses/{warehouseId}/products	Get All Products	▼
GET	/api/warehouses/{warehouseId}/products/{productId}	Get Product By Id	▼
PATCH	/api/warehouses/{warehouseId}/products/{productId}	Update Product Details	▼
PUT	/api/warehouses/{warehouseId}/products/{productId}	Update Or Create Product	▼
DELETE	/api/warehouses/{warehouseId}/products/{productId}	Delete Product	▼
Stock Management			^
GET	/api/warehouses/{warehouseId}/inventory	Get Inventory	▼
GET	/api/warehouses/{warehouseId}/inventory/{productId}	Get Product Inventory	▼
POST	/api/warehouses/{warehouseId}/inventory/{productId}/increase	Increase Stock	▼
POST	/api/warehouses/{warehouseId}/inventory/{productId}/decrease	Decrease Stock	▼
POST	/api/warehouses/{warehouseId}/inventory/{productId}/transfer	Transfer Stock	▼

3 Tehnologii și Tool-uri utilizate

Python & FastAPI - Dezvoltarea microserviciului de backend și a logicii de business.

React.js & Vite - Construirea interfeței frontend și gestionarea stării aplicației.

PostgreSQL (Supabase) - Stocarea persistentă a datelor într-o bază de date relațională în cloud.

SQLAlchemy - ORM utilizat pentru maparea obiectelor **Python** pe tabelele bazei de date.

Tailwind CSS - Stilarea rapidă și responsivă a componentelor frontend.

Render - Găzduirea live a **API**-ului de backend.

Hostinger - Deployment-ul versiunii de producție a frontend-ului.

Visual Studio Code - Mediul principal de dezvoltare (**IDE**).

Swagger (OpenAPI) - Generarea automată a documentației interactive pentru testarea și vizualizarea endpoint-urilor **API** în timp real.

Pydantic - Definirea structurilor de date (schemas) și validarea automată a input-urilor pentru a asigura integritatea bazei de date.



4 Structura aplicației

4.1 Schema bazei de date (Database schema)

Sistemul este construit pe o arhitectură relațională:

- **Produse (Products):** Stochează detalii despre SKU, preț și denumire.
- **Depozite (Warehouses):** Monitorizează locația și capacitatea maximă.
- **Furnizori (Suppliers):** Gestionează baza de date a partenerilor de la care provin produsele, incluzând date de contact.



4.2 Structura backend-ului (Module)

- **main.py**: Punctul de intrare și configurarea **CORS**.
- **database.py**: Gestionarea conexiunii către instanța Supabase.
- **models.py**: Definirea structurii tabelelor PostgreSQL.
- **Schemas.py**: Validarea datelor de intrare/ieșire folosind **Pydantic**.

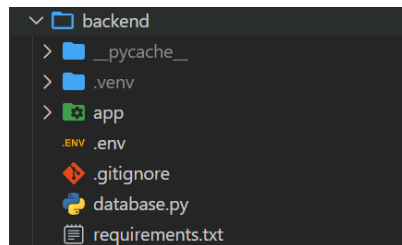
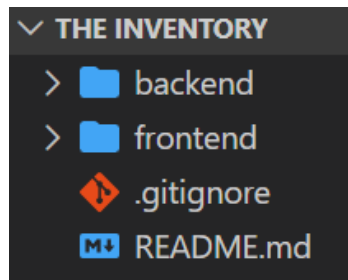
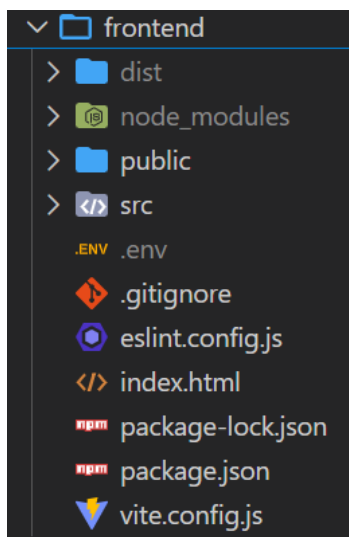
4.3 Structura frontend-ului (Frontend app structure)

Interfața este construită în **React.js**, fiind organizată pe componente reutilizabile:

- **Componente de UI**: Dashboard, Tabele de inventar, Formulare de editare.
- **State management**: Gestionarea datelor primite de la API-ul de pe **Render** prin hook-uri **React** (**useEffect**, **useState**).
- **Design**: Implementat cu **Tailwind CSS** pentru un aspect modern și responsiv.

4.4 Structura repo-ului Git

- **/backend**: Conține codul **Python**, fișierul **requirements.txt** și logica **API**.
- **/frontend**: Conține proiectul **React**, configurarea **Tailwind** și folderele de componente.
- **.gitignore**: Extensiile fișierelor ignorate de **Git**.
- **README.md**: Documentația de bază a repository-ului.



5 Configurare și rulare

5.1 Rulare locală (Development):

```
backend > requirements.txt
1 annotated-doc==0.0.4
2 annotated-types==0.7.0
3 anyio==4.13.0
4 certifi==2026.2.25
5 charset-normalizer==3.4.6
6 click==8.3.1
7 colorama==0.4.6
8 fastapi==0.135.2
9 greenlet==3.3.2
10 h11==0.16.0
11 idna==3.11
12 psychopg2-binary==2.9.11
13 pydantic==2.12.5
14 pydantic_core==2.41.5
15 python-dotenv==1.2.2
16 requests==2.33.1
17 SQLAlchemy==2.0.48
18 starlette==1.0.0
19 typing-inspection==0.4.2
20 typing_extensions==4.15.0
21 urllib3==2.6.3
22 uvicorn==0.42.0
```

- **Backend:** Crearea unui mediu virtual, instalarea dependențelor (`pip install -r requirements.txt`) și pornirea serverului cu:

`.\venv\Scripts\activate`

`uvicorn app.main:app --reload.`

```
(.venv) PS C:\Users\struf\Desktop\SITE\The Inventory\backend> .\venv\Scripts\activate
(.venv) PS C:\Users\struf\Desktop\SITE\The Inventory\backend> uvicorn app.main:app --reload
INFO: Will watch for changes in these directories: ['C:\\Users\\struf\\Desktop\\SITE\\The
']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [20228] using StatReload
INFO: Started server process [22388]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

- **Frontend:** Instalarea modulelor **Node** (`npm install`) și pornirea serverului de dezvoltare (`npm run dev`).

```
PS C:\Users\struf\Desktop\SITE\The Inventory\frontend> npm run dev

> the-inventory-web@0.0.0 dev
> vite

Re-optimizing dependencies because vite config has changed

VITE v5.4.21 ready in 2408 ms

→ Local:   http://localhost:5173/The-inventory/
→ Network: use --host to expose
→ press h + enter to show help
```

5.2 Rulare producție:

- **Backend:** este conectat automat la **GitHub** prin **Render** pentru **Continuous Deployment**.
- **Frontend:** este compilat (`npm run build`) și urcat pe **Hostinger**.



6 Concluzii / Lecții învățate

Prin dezvoltarea acestui proiect am reușit să aprofundez mecanismele de gestionare a datelor, să învăț cum să creez o infrastructură scalabilă bazată pe **API-uri** și să dezvolt **Full-Stack**. Cele mai importante cunoștințe dobândite:

- **Modelarea datelor:** Am implementat o structură relațională în **PostgreSQL** care asigură consistența

informațiilor între produse, depozite și furnizori, gestionând logic stocurile fără a supraîncărca baza de date cu tabele redundante.

- **Dezvoltarea microserviciilor:** Am utilizat **FastAPI** pentru a construi un **API** asincron, punând accent pe validarea datelor prin **Pydantic** și pe securizarea rutelor de transfer de stoc.
- **Gestiunea resurselor în cloud:** Am coordonat un mediu distribuit, învățând să interconectez o bază de date securizată în cloud (**Supabase**) cu un serviciu de hosting pentru **API** (**Render**).